27,350,000 token

彻底搞懂

训 agent 的工程坑

全长 29 分钟 | 建议先收藏

强化学习系统：三块内容

1

总览

一个训练步 · 两套引擎

2

agent 强化学习的坑

光看公式预料不到

3

系统设计的选择

框架有哪几种搭法



课件 p.2

强化学习系统：三块内容

1

总览

一个训练步 · 两套引擎

2

agent 强化学习的坑

光看公式预料不到

3

系统设计的选择

框架有哪几种搭法

修 bug · 编程 agent

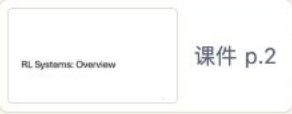
retry.py

```
def retry_count(config):
    return config.get("retries") or 3
```

验证器 · 四个测试（输入 → 期望输出）

输入	缺失	0	None	2
期望	3	0	3	2





课件 p.2

总览
agent 强化学习的坑
系统设计

策略梯度更新：框架要支持哪些操作?

optim_step　backward　forward　采样也是前向

$$\theta \mathrel{+}= \Sigma_t \left(\mathrm{R}(\tau) - b(h_t)\right) \nabla_\theta \log \pi_\theta(a_t \mid h_t)$$

env.verify　env.step　+ agent harness

大白话

对轨迹里每一步动作 → 算它的概率往哪个方向变 → 乘上这条轨迹比基线好多少 → 加起来，参数往这个方向挪一步

问题是：要训这样的模型，框架最核心的操作有哪些?



课件 p.3（中文重绘）

总览
agent 强化学习的坑
系统设计

为什么不直接在 PyTorch 里手写?

optim_step　　backward　　forward

$$\theta \mathrel{+}= \Sigma_t \left(\mathrm{R}(\tau) - b(h_t)\right) \nabla_\theta \log \pi_\theta(a_t \mid h_t)$$

PyTorch 接口

optimizer.step()　　model.backward()　　model.forward()

	SkyRL	Miles	veRL	PrimeRL
环境和 rollout 那一层	确实各家不同			
前向 · 反向 · 优化器	forward backward optim_step 在哪个框架里都差不多			

?

复杂在哪?



课件 p.4（中文重绘）

采样和训练，不共用一个前向

$$\theta \mathrel{+}= \Sigma_t \left(\mathrm{R}(\tau) - b(h_t)\right) \nabla_\theta \log \pi_\theta(a_t \mid h_t)$$

optim_step　forward_backward　sample

推理引擎

sample

例如 vLLM、SGLang

解码为主 · 一个接一个

不要梯度 → 显存放 KV 缓存

显存 | KV 缓存

明明是同一个模型
为什么要分开?

训练引擎

forward_backward

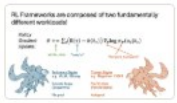
例如 Megatron、FSDP2

预填充式 · 整条并行

自动求导 → 留着激活和计算图

显存 | 激活和计算图

示意



课件 p.5（中文重绘）

分开之后，各做各的优化

推理引擎

`sample`

例如 vLLM、SGLang

- KV 缓存
- 前缀缓存
- 连续批处理
- 推测解码
- 模型切分 PP · CP · TP · EP 等

把推理和训练**拆开**才能分别优化

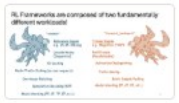
后面挑着讲

训练引擎

`forward_backward`

例如 Megatron、FSDP2

- 激活检查点
- 前缀共享
- 批内样本打包 本讲不展开
- 模型切分 PP · CP · EP 等



课件 p.6（中文重绘）

总览
agent 强化学习的坑
系统设计

两套引擎之间的来往

推理引擎

`sample`

例如 vLLM、SGLang

轨迹数据 →

← 权重同步

训练引擎

`forward_backward`

例如 Megatron、FSDP2

推理里的策略 ≈ 训练里的策略

同样的权重，略有不同的计算

第 11 讲 · 重要性采样



课件 p.7（中文重绘）

把一个训练步拆开：五格

修 bug · 编程 agent

retry.py

```
def retry_count(config):
    return config.get("retries") or 3
```

验证器 · 四个测试（输入 → 期望输出）

输入	缺失	0	None	2
期望	3	0	3	2

1 读 改 测 再改 0
2 读 改 再改 1
3 读 改 测 测 再改 0
4 读 改 测 再改 0
5 读 改 再改 1
6 读 改 测 测 测 测 测 测 再改 0 最长
7 读 改 测 再改 1
8 读 改 测 测 再改 0

整条并行重算 · 留着激活值

组内优势加权 → 求梯度



训练的切法 → 推理的切法

示意

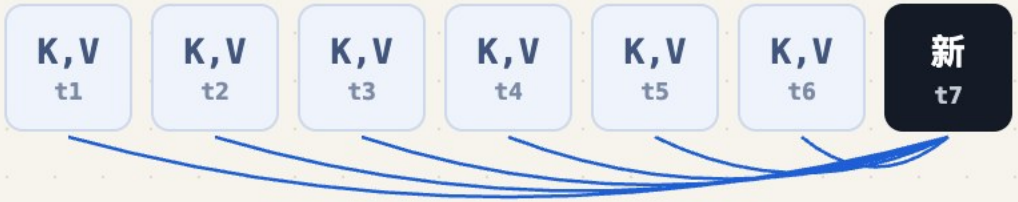




课件 p.8（中文重绘）

KV 缓存：拿显存换计算

KV 缓存 · 每个过去的 token 一格



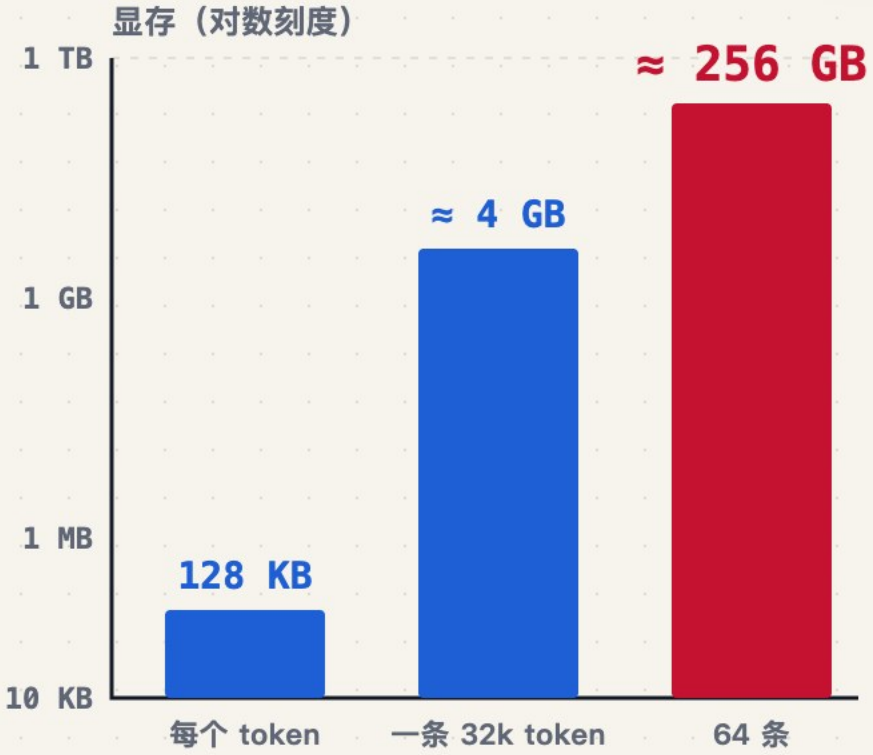
t7 只算自己的 q、k、v，再和缓存里的 K、V 做注意力

~~不缓存：每出一个 token，前缀重新预填充一遍~~

拿大量显存，换掉大量重复计算

训练和推理合成一个引擎时，显存怎么分 示意





Llama-3-8B · bf16

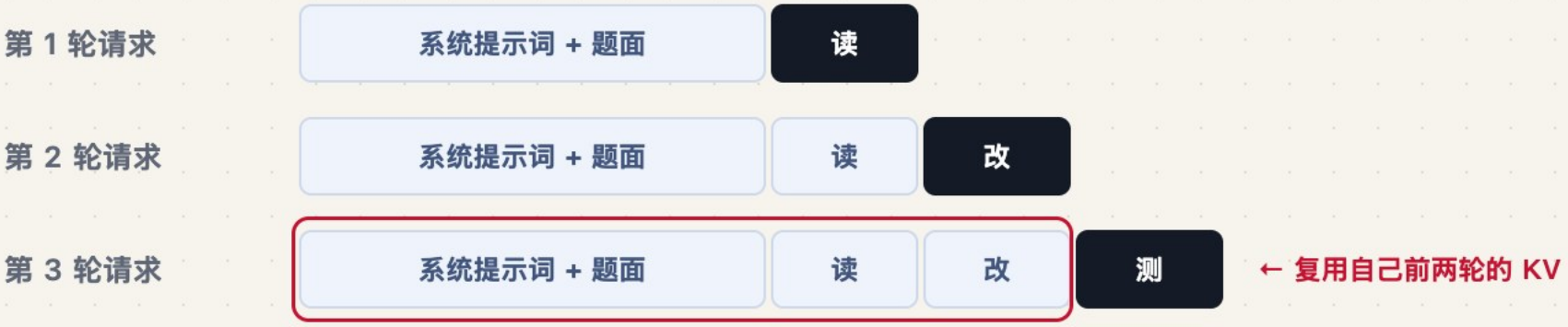
每 token = 2 × 32 层 × 8 个 KV 头 × 128 维 × 2 字节



课件 p.9

前缀缓存：请求之间也复用

第 1 条轨迹：每一步都是一次单独的模型请求



不做缓存 → 大量重复计算

SGLang · RadixAttention

请求之间复用前缀的 KV

vLLM · 自动前缀缓存

请求之间复用前缀的 KV



课件 p.10

● 总览
agent 强化学习的坑系统设计

连续批处理：空位马上补上

静态批处理

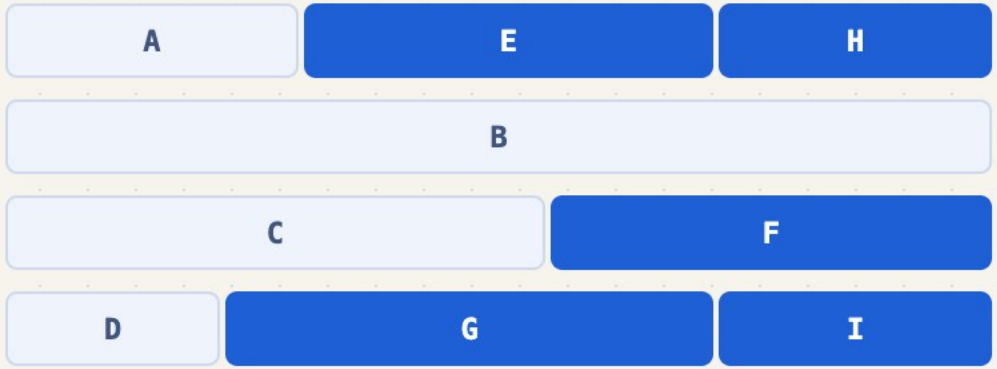
示意



另外 7 个位置都在等第 6 条

长短不一 → 大量空闲

连续批处理



跑完的马上离开，空位下一步就补上

排队：E F G H I

Orca · Yu et al. 2022



课件 p.11

连续批处理：空位马上补上

解码被挡住

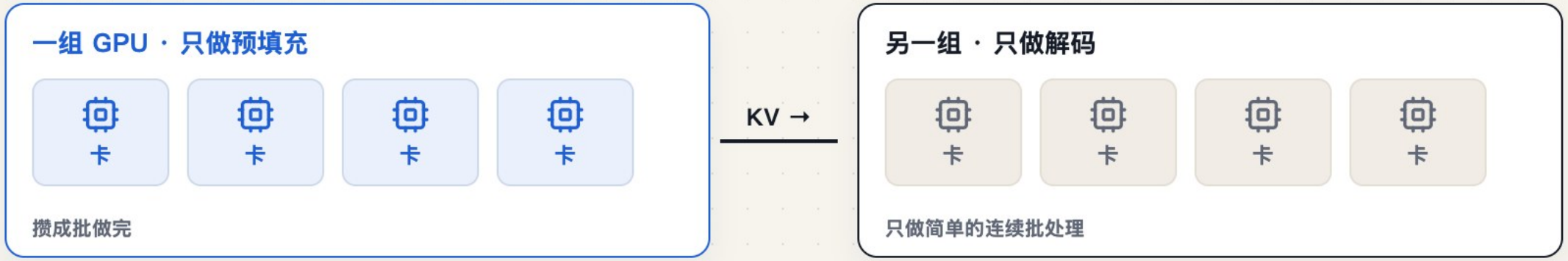
一整块预填充



预填充：整段提示词并行处理，是另一种负载

分块预填充





Orca · Yu et al. 2022



课件 p.11

总览
agent 强化学习的坑
系统设计

推测解码：小模型先猜，策略模型一次检查

在线训

草稿模型

模型自带的草稿模块
或单独的小模型

→ 连着猜几个 token → t1 t2 t3 t4 →

策略模型

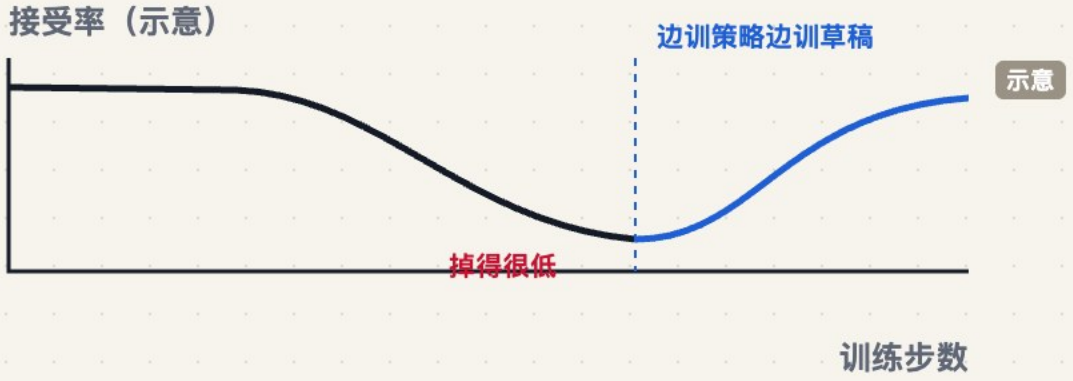
一次并行地检查

→ 接受 t1 | 接受 t2 | 拒绝 t3

t3′ 从这里重新采

拒绝采样：结果和直接从策略模型采样完全等价

辅助目标：监督微调 / 蒸馏



策略在变，
草稿模型也要跟着训

Leviathan et al. 2023



课件 p.12

总览
agent 强化学习的坑
系统设计

激活检查点：用重算换显存

训练这边：多算一点、少用显存



显存

每层一个输入 + 一层的激活

代价

全量重算 约 +33% 计算

选择性重算（只重算注意力）便宜得多

前向 1 份 + 反向约 2 份 → 多 1 份前向 ≈ 多三分之一

Chen et al. 2016 · Korthikanti et al. 2022



课件 p.13

总览
agent 强化学习的坑
系统设计

前缀共享：题面只算一遍

题面 + 代码 + 测试说明



~~稠密：题面在 8 个数据点里各算一遍前向和反向~~

共享：题面只算一遍

分支之间互相看不见 · 位置编号和单独成序列时一样

8 个分支的梯度 → 都加回到题面那一段



课件 p.14

两组实测：口径要分清

ZoRRo · Snowflake

重复的提示词 token（占比）

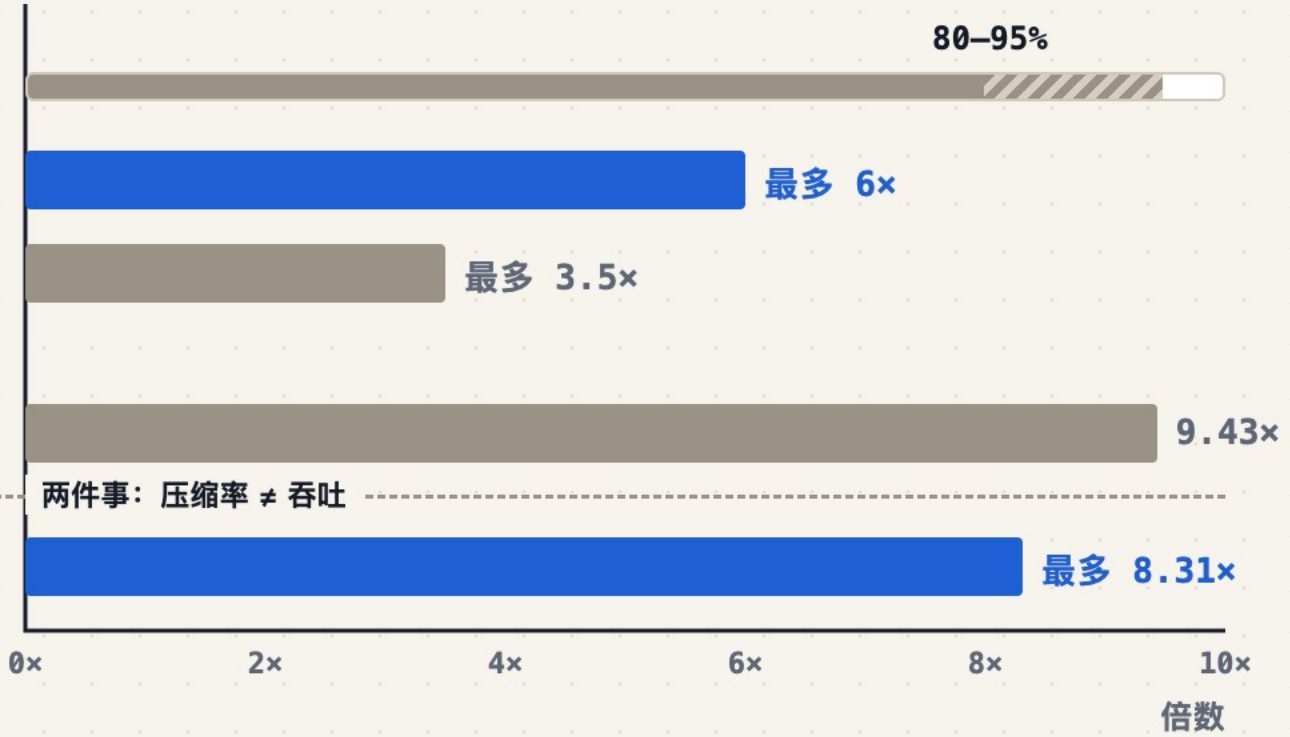
策略更新那一步

端到端训练

AReaL-DTA · 动态树注意力

前缀树压缩率（token 数）

训练吞吐（对比稠密训练）



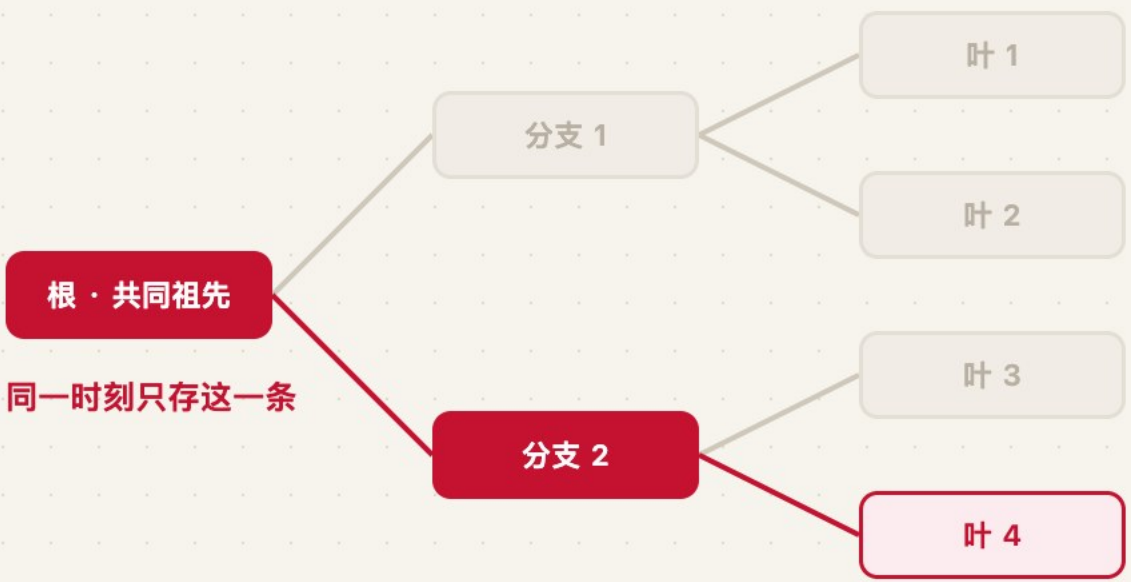
Snowflake 2026 · Zhang et al. 2026



课件 p.14

总览
agent 强化学习的坑
系统设计

深度优先：只存一条路径



示意

① 共同祖先：前向

② 往下走一支：前向 → 马上反向

③ 做完把这段释放

④ 再回去走另一支

存着的激活，从来不超过
一条从根到叶的路径

训练那边
关心反向和激活

推理那边
主要操心 KV 缓存放不放得下



课件 p.14

并行：一张卡放不下，就切开

总览
agent 强化学习的坑
系统设计

1 数据并行 示意

全局批 → 小批



每张卡一份完整模型

全局批切成小批，各算各的

小批按 token 数装箱，比如 4 万

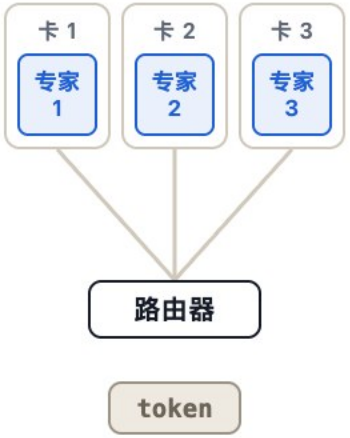
2 流水线并行 示意



不同的层放在不同的卡上

后一张算小批 1 时，前一张已在算小批 2

3 专家并行

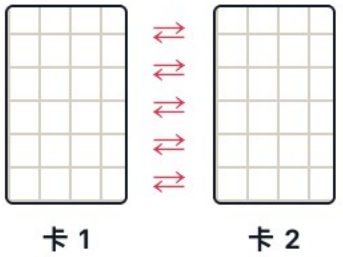


专家分到不同的卡上

一张卡装不下整个模型，装得下一两个专家

4 张量并行

一个矩阵运算



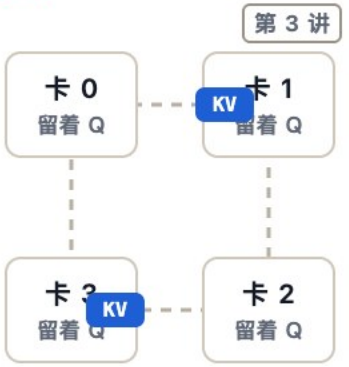
通信多

只在一台机器内

矩阵运算拆成几小块

运算互相依赖，通信很多

5 上下文并行



K、V 块转圈传

每张卡负责一段序列

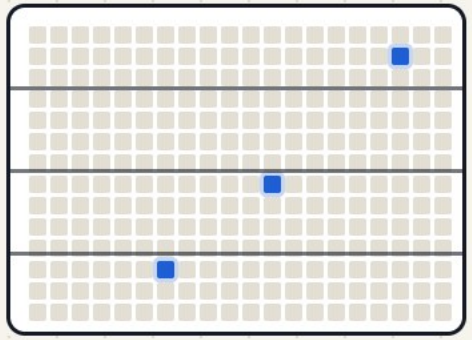
把自己的 K 和 V 一张张往下传



课件 p.15

总览
agent 强化学习的坑
系统设计

权重同步：两个 TB，还是几十 GB



约 2 TB —万亿参数 · bf16

约 99% 的参数在 bf16 下一位都没变（PULSE）

全量广播给每一个推理 rank

只同步变化的那部分

跨集群：传到云存储的桶再下载 ✕

推理 rank 1

推理 rank 2

推理 rank 3

推理 rank 4

按推理引擎的并行布局重新切分

每次同步的数据量（对数刻度）



论文实验做到 7B：14 GB → 约 108 MB

PULSE · Miahi & Belilovsky 2026

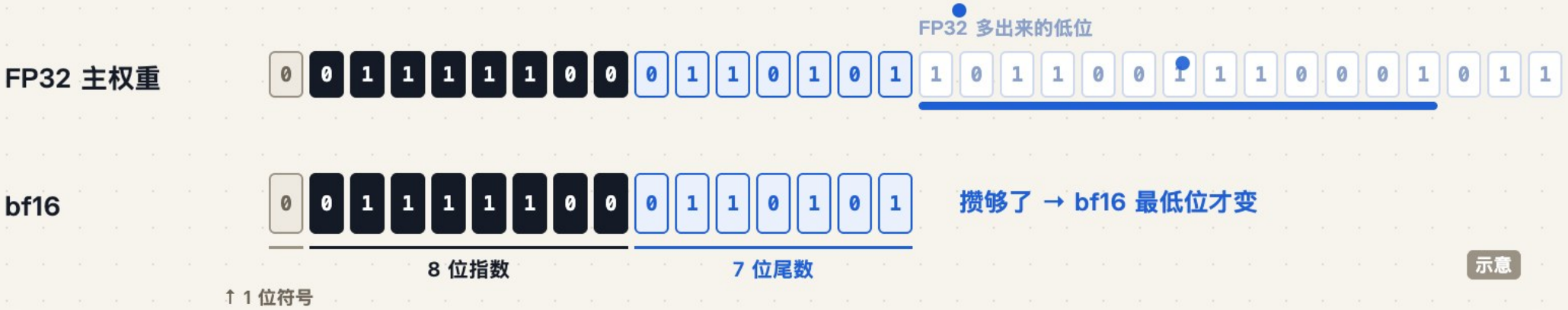


课件 p.16

总览
agent 强化学习的坑
系统设计

课堂问答：为什么九成九不变

课上有人问 **九成九的参数不变，是因为 bf16 吗？** 是的



bf16 的尾数只有 7 位

每条轨迹给的监督很少，尤其只看最后奖励

很多更新小得在 bf16 下根本看不出来

小更新在 FP32 主权重里累积 我理解是

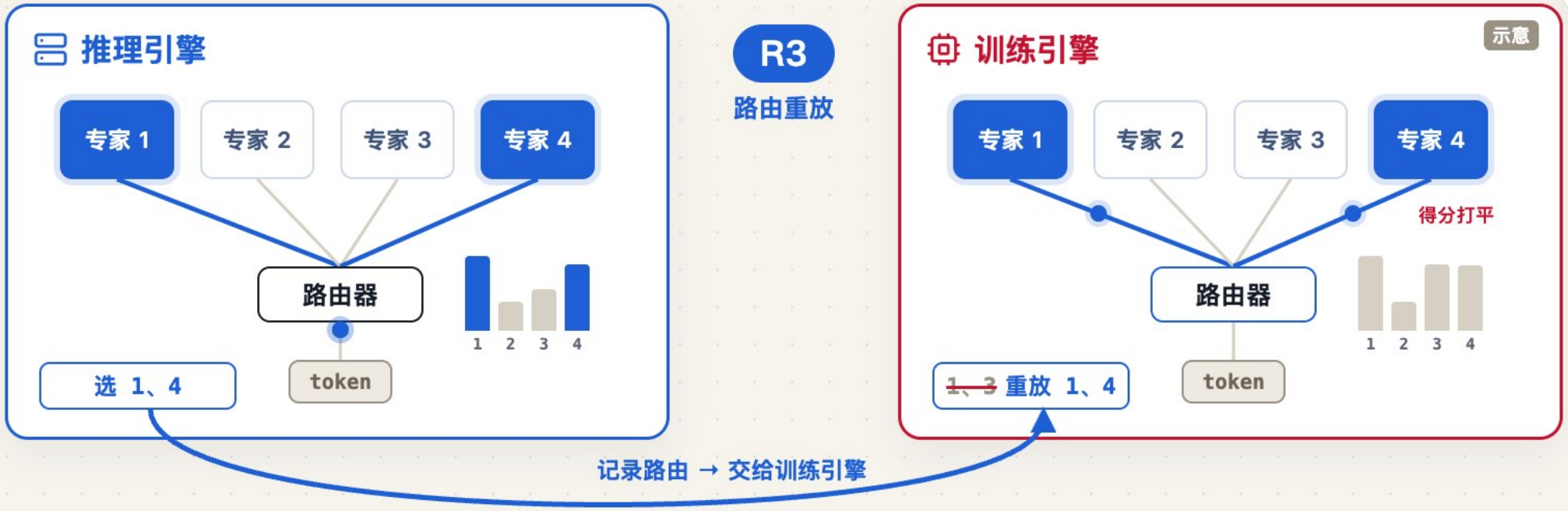
混合精度训练里，优化器手上有一份 FP32

攒够了，bf16 的那份才变



课件 p.16 · 课上问答

MoE 路由：两边可能选了不同的专家



对混合专家模型的训练稳定性帮助很大

损失、奖励不涨，先查查有没有开 R3



路由记录

示意

Ma et al. 2025 · R3



课件 p.17

总览
• agent 强化学习的坑
系统设计

序列延展：两个 harness

Harness A

✓ 答案是 A

一直追加到历史后面

第 1 轮 | T | A_1 | O_1 |

第 2 轮 | T | A_1 | O_1 | A_2 | O_2 |

第 3 轮 | T | A_1 | O_1 | A_2 | O_2 | A_3 | O_3 |

第 4 轮 | T | A_1 | O_1 | A_2 | O_2 | A_3 | O_3 | A_4 | O_4 |

T 任务 A 动作 O 观察 S 压缩后的交互历史

能避免的话，A 对你更好

课上投票

多数人选 B

Harness B

每两个观察压缩一次

第 1 轮 | T | A_1 | O_1 |

第 2 轮 | T | A_1 | O_1 | A_2 | O_2 |

O_2 之后：保留 T，把 A_1 O_1 A_2 O_2 总结成 S

第 3 轮 | T | S | A_3 | O_3 |

第 4 轮 | T | S | A_3 | O_3 | A_4 | O_4 |

生产环境常见：Codex Claude Code OpenHands 都会压缩上下文

序列实在太长时，有时不得不训 B



课件 p.19

拼进训练批：是什么样

Harness A：一个拼起来的数据点

整条轨迹一个优势 Â，乘在每个动作上

token 段	任务	A_1	O_1	A_2	O_2	A_3	O_3	A_4	O_4
损失掩码	0	1	0	1	0	1	0	1	0
优势	0	Â	0	Â	0	Â	0	Â	0

Harness B：两个数据点，都带任务

token 段	任务	A_1	O_1	A_2	O_2
损失掩码	0	1	0	1	0
优势	0	Â	0	Â	0

任务	S	A_3	O_3	A_4	O_4
0	0	1	0	1	0
0	0	Â	0	Â	0

任务提示词重复了：预填充 A 只做一遍，B 做**两遍** · 多出来的全是重复计算

任务 · 算了两遍



课件 p.20

总览
• agent 强化学习的坑
系统设计

修 bug 的轨迹：压缩一次，就拆成两条



读 改 测 再改 0

第三轮测试失败后，压缩一次

数据点 1



读 改 测

题面和那几行代码，训练端算两遍

数据点 2



S 再改 0

动态树注意力：能补救一部分

示意

能保住完全一样的前缀，就一直往下拼

序列延展：每一轮的输入，都以上一轮的完整输出开头



课件 p.20

总览
• agent 强化学习的坑
系统设计

默认的聊天模板：悄悄删掉历史里的推理

Qwen3 · Qwen3.5 默认模板，会把历史里的推理删掉

第 1 轮

任务 → 第 1 轮 思考 + 第 1 轮 工具调用

对上了：前缀保住

第 2 轮

任务 + 第 1 轮 思考 + 第 1 轮 工具调用 + harness 输出 → 第 2 轮 思考 + 第 2 轮 工具调用

修 bug 的 agent · 第 1 轮思考

0 是假值，or 会把它换成 3

然后调工具：读代码

渲染器

Tinker

PrimeRL

跨轮保留推理

完全不要推理

目的：把前缀保住

给每个模型一套定制的渲染器



课件 p.22

轨迹用什么格式，交给训练引擎？

总览
• agent 强化学习的坑
系统设计

推理引擎

`sample`

例如 vLLM、SGLang

`[ token … ]`

轨迹数据 →

← 权重同步

训练引擎

`forward_backward`

例如 Megatron、FSDP2

↻ ~~再重新分词~~

~~① 字符串~~

聊天消息　提示词工程　解析模型输出

② token

直接存 token、传 token，不重新分词

TITO token 进、token 出



课件 p.24

总览
• agent 强化学习的坑
系统设计

原因一：字符串上一点小变换

推理端 · 模型生成（紧凑写法）

工具调用 · JSON

```
{"retries":0}
```

训练端 · 模板重新渲染

工具调用 · JSON

```
{"retries": 0}
```

推理端采的 token	{"	retries	":0	}
训练端重新分词	{"	retries	":	0	}

示意

← 从第三格开始对不上

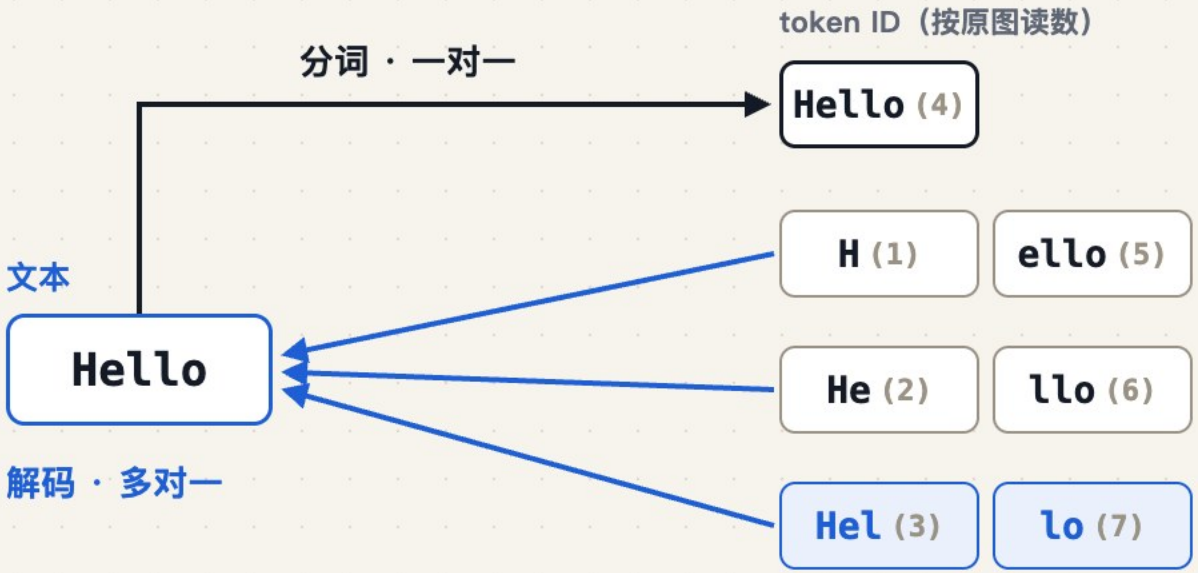
训的是模型从没采过的 token 又离策略了



课件 p.24

原因二：分词一对一，解码多对一

总览
• agent 强化学习的坑
系统设计





模型采的 Hel lo ≠ 训练端重新分词 Hello

现在：记下确切的 token 交给训练引擎



课件 p.24

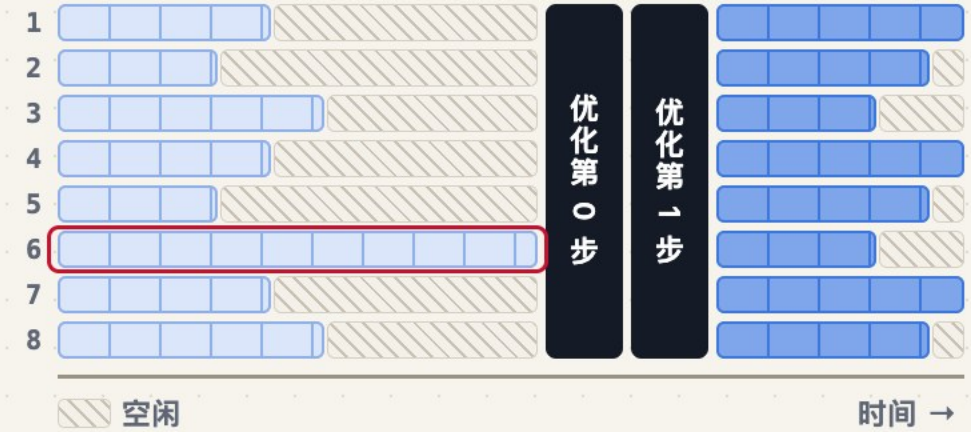
总览
• agent 强化学习的坑
系统设计

同步共用卡，异步分开卡

只要能容忍一点离策略、做好修正，就能让推理和训练重叠起来

同步

第 6 条最长 → 其他位置全在等



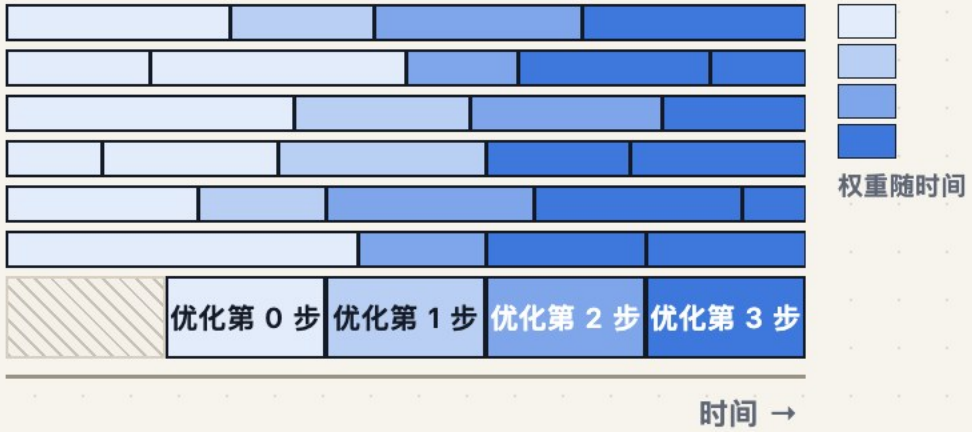
示意



8 张卡 · 两个引擎轮流用

训练引擎睡觉 → 拉起推理引擎

异步



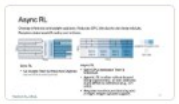
瓶颈常常在 rollout → 一般给 rollout 多分



六个节点推理

两个节点训练

3 : 1 课上举的一个真实例子



课件 p.25（中文重绘）

总览
agent 强化学习的坑
系统设计

生成到一半换权重

异步的流程靠两样东西：连续批处理 + 生成到一半换权重



SGLang 的暂停生成接口 `POST /pause_generation` 三种模式

abort

直接把请求退回去

in_place

原地暂停，留着 KV 缓存

retract

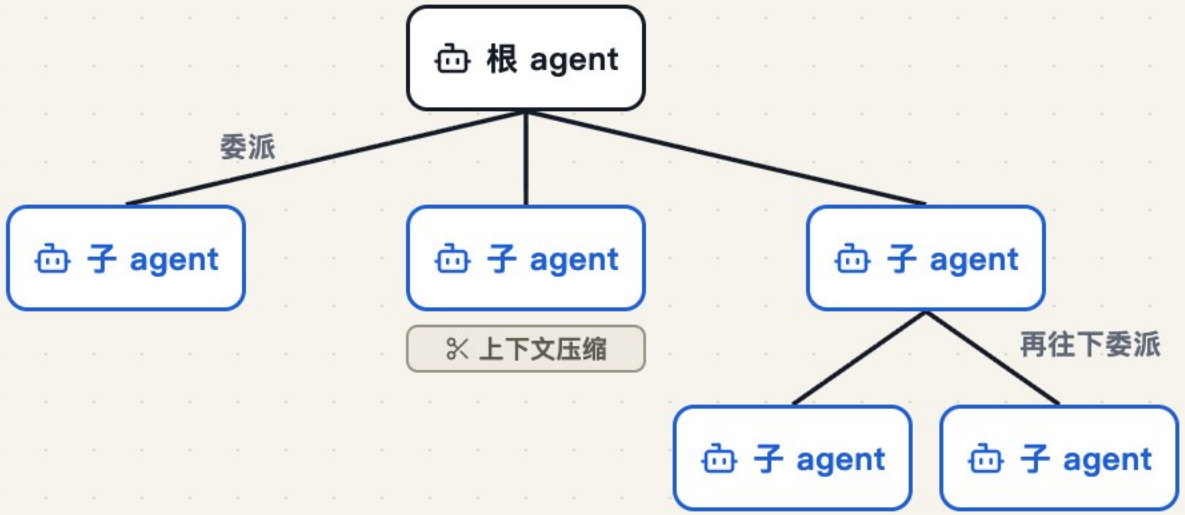
撤回等待队列，KV 缓存可以清掉，换上新模型后重算

第 11 讲 · 离策略修正



课件 p.26（中文重绘）

子 agent 一层层委派



示意

来自递归语言模型那篇论文

→ 后来拿它训递归的多 agent 系统

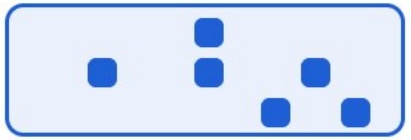
子 agent　上下文压缩

→ 推理的组织方式变得很复杂

agent 强化学习 ≠ 传统的推理强化学习

和上下文压缩把一个数据点拆成几个，是同一类问题

整棵树的 rollout 算一条



→ 一条

?

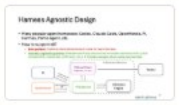
每个子 agent 的轨迹各算一条



课件 p.27（示意重绘）

怎么支持各种 harness：放一个中间人代理

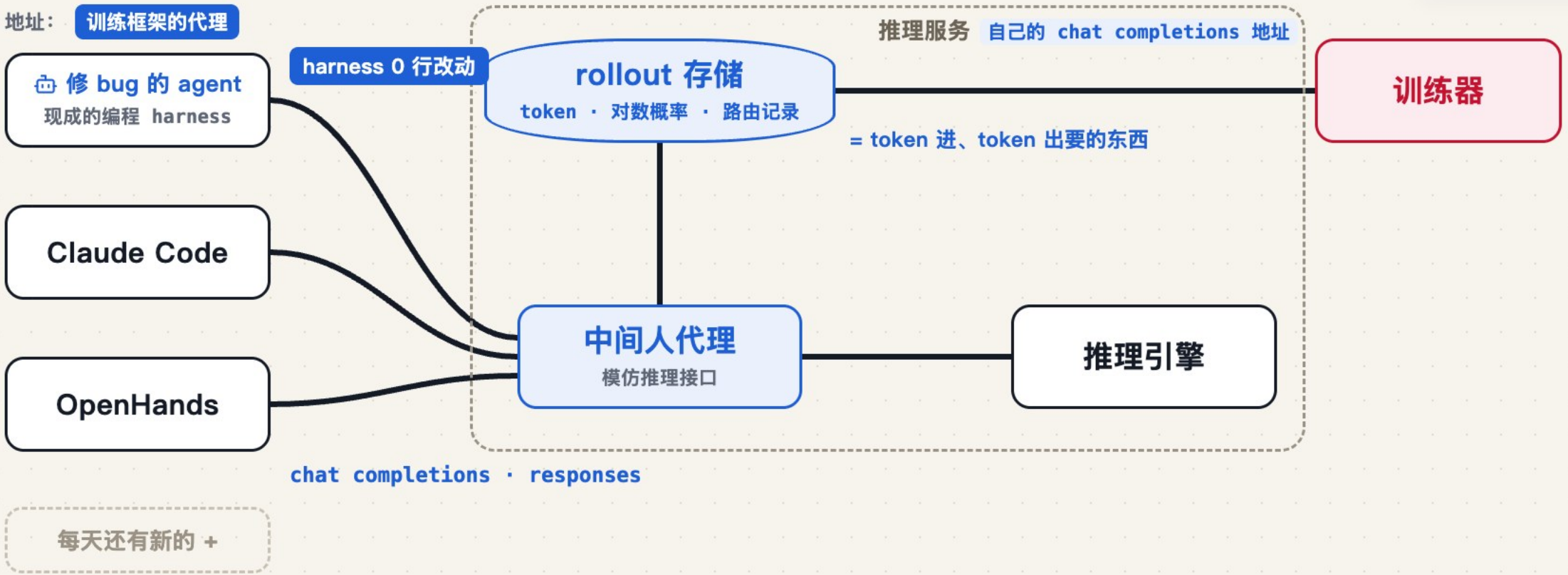




课件 p.29（中文重绘）

总览
agent 强化学习的坑
● 系统设计

怎么支持各种 harness：放一个中间人代理



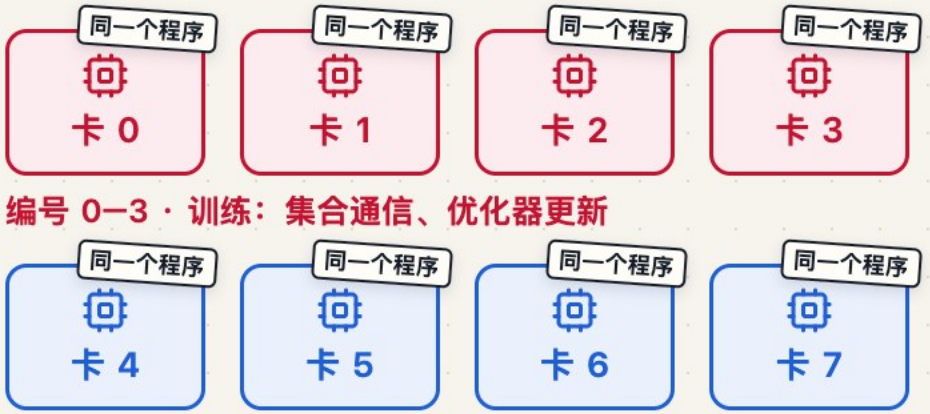


课件 p.29（中文重绘）

第一种：单程序多数据（SPMD）

每张卡都跑这一份 · 简化伪代码

```
rank = dist.get_rank()
train = dist.new_group([0, 1, 2, 3])
infer = dist.new_group([4, 5, 6, 7])
if rank < 4:
    while True:
        batch = next_batch_shard(train)
        forward_backward(batch, train)
        optimizer.step()
        publish_weights_async(train)
else:
    while True:
        refresh_weights_if_ready(infer)
        prompt = next_prompt(infer)
        trace = rollout(prompt, infer)
        enqueue_trajectory(trace, infer)
```



所有协调都写在这一个程序里

✓ 一个程序装下一切，一目了然，想改就改

× 可它非常不灵活

AReaL 1.0 版本以前就是这样

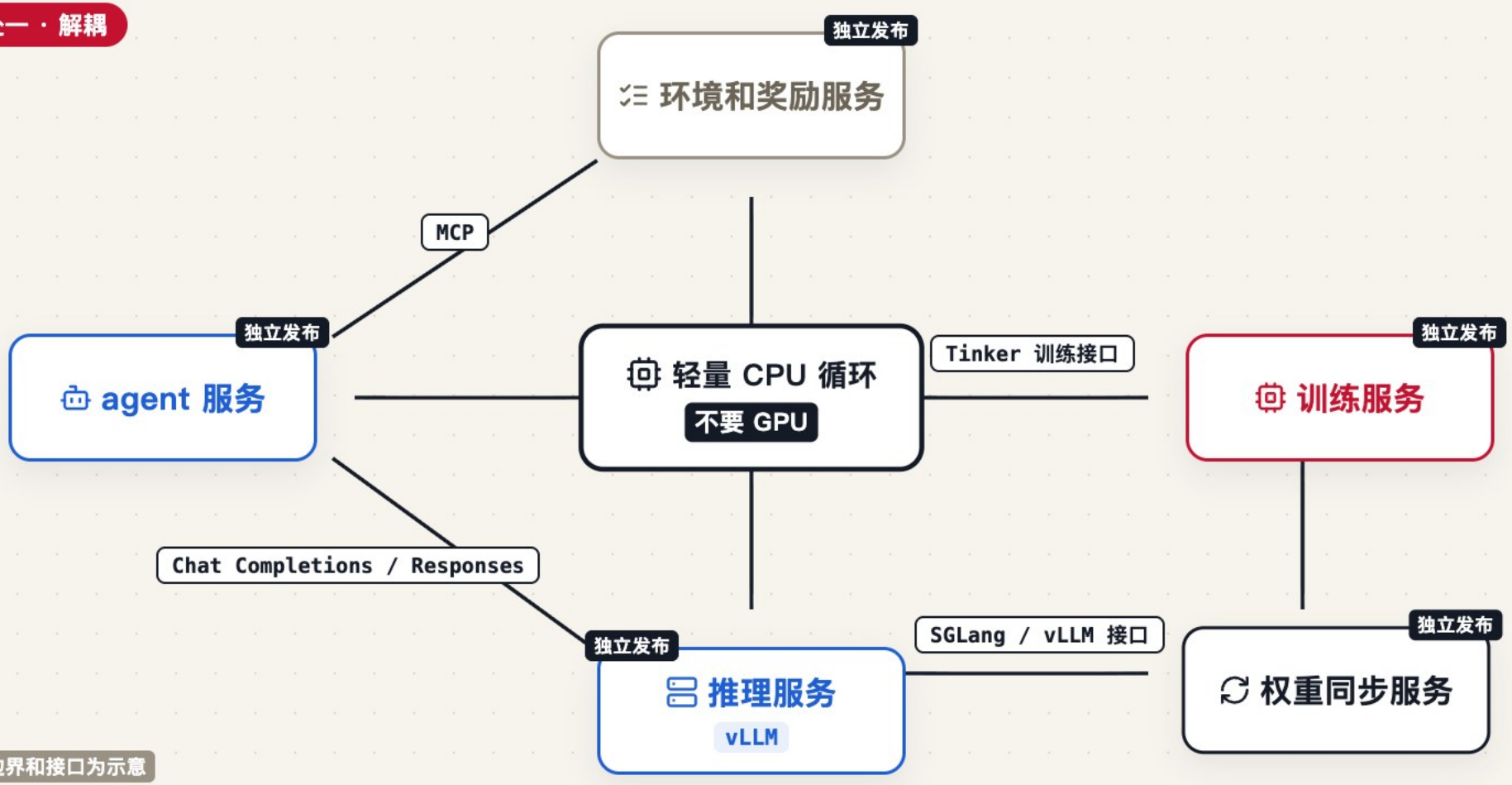


课件 p.30（中文重绘）

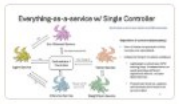
总览
agent 强化学习的坑
系统设计

第二种：一切皆服务，加一个单一的控制器

好处一 · 解耦



服务边界和接口为示意

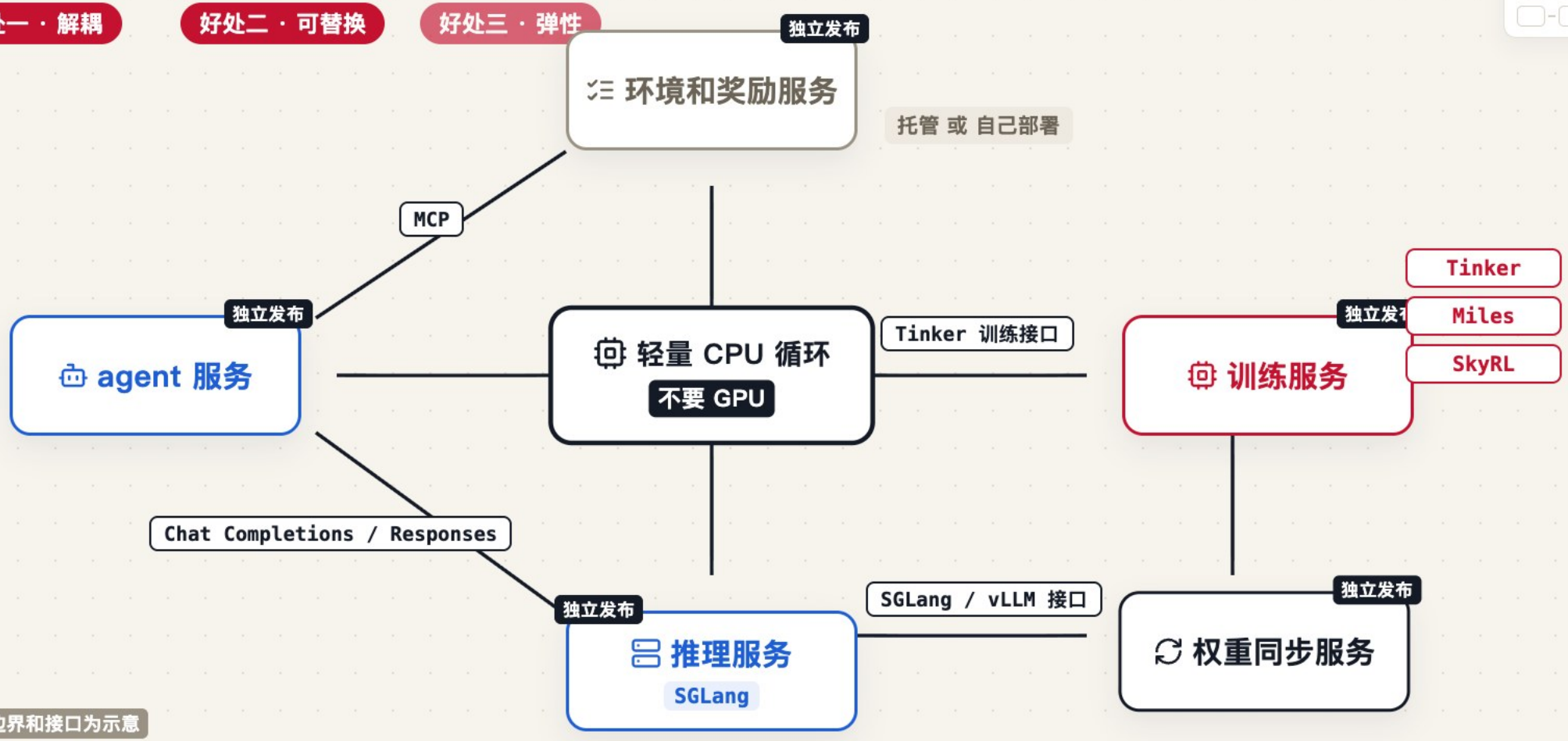


课件 p.32（中文重绘）

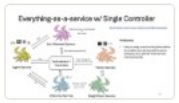
总览
agent 强化学习的坑
系统设计

第二种：一切皆服务，加一个单一的控制器

好处一 · 解耦　好处二 · 可替换　好处三 · 弹性



服务边界和接口为示意

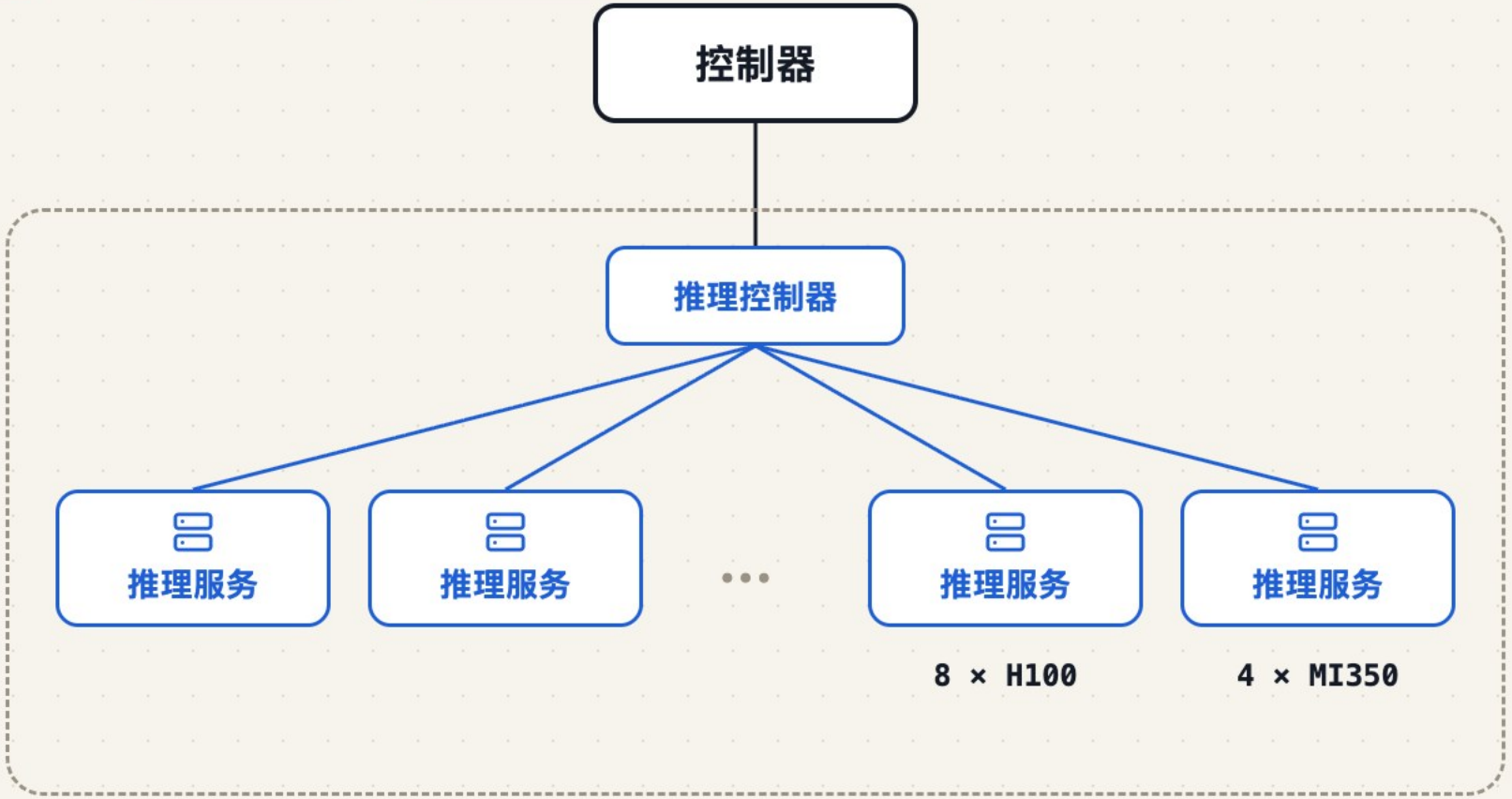


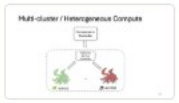
课件 p.33（中文重绘）

总览
agent 强化学习的坑
系统设计

第二种：一切皆服务，加一个单一的控制器

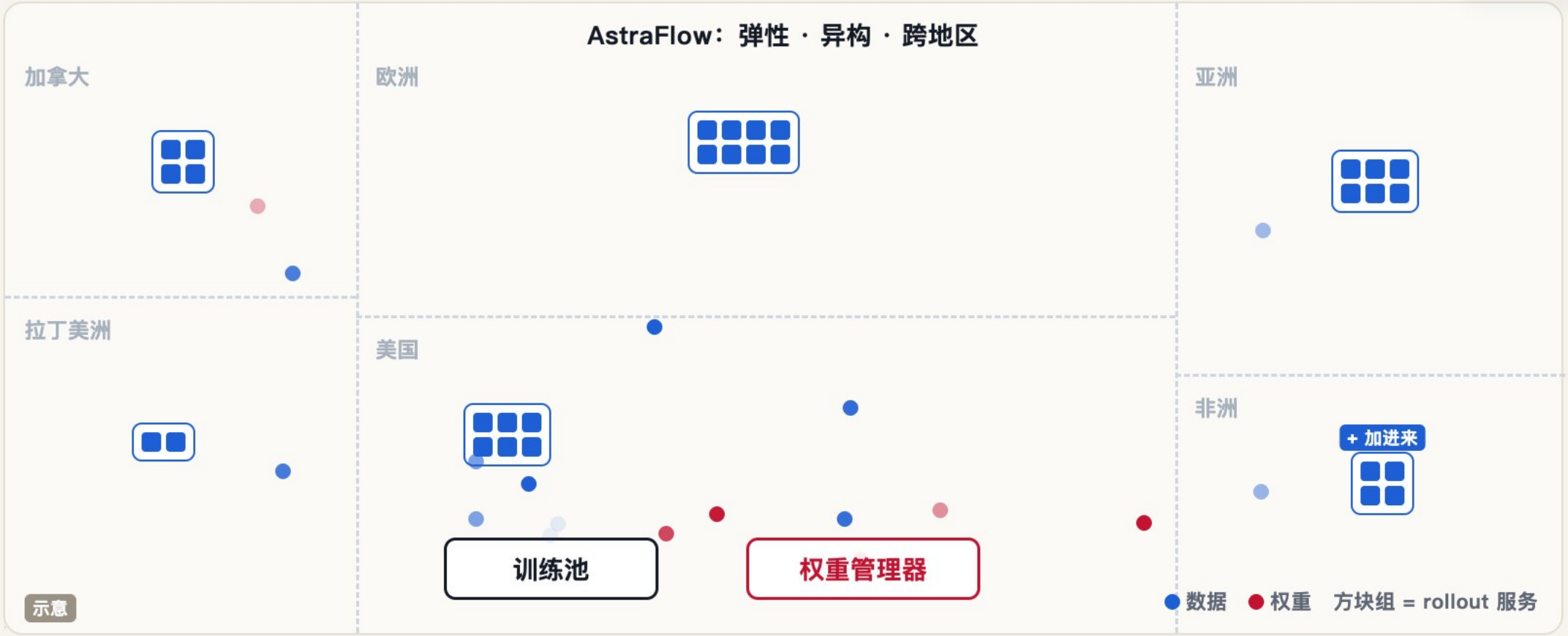
好处一 · 解耦　好处二 · 可替换　好处三 · 弹性





课件 p.36（中文重绘）

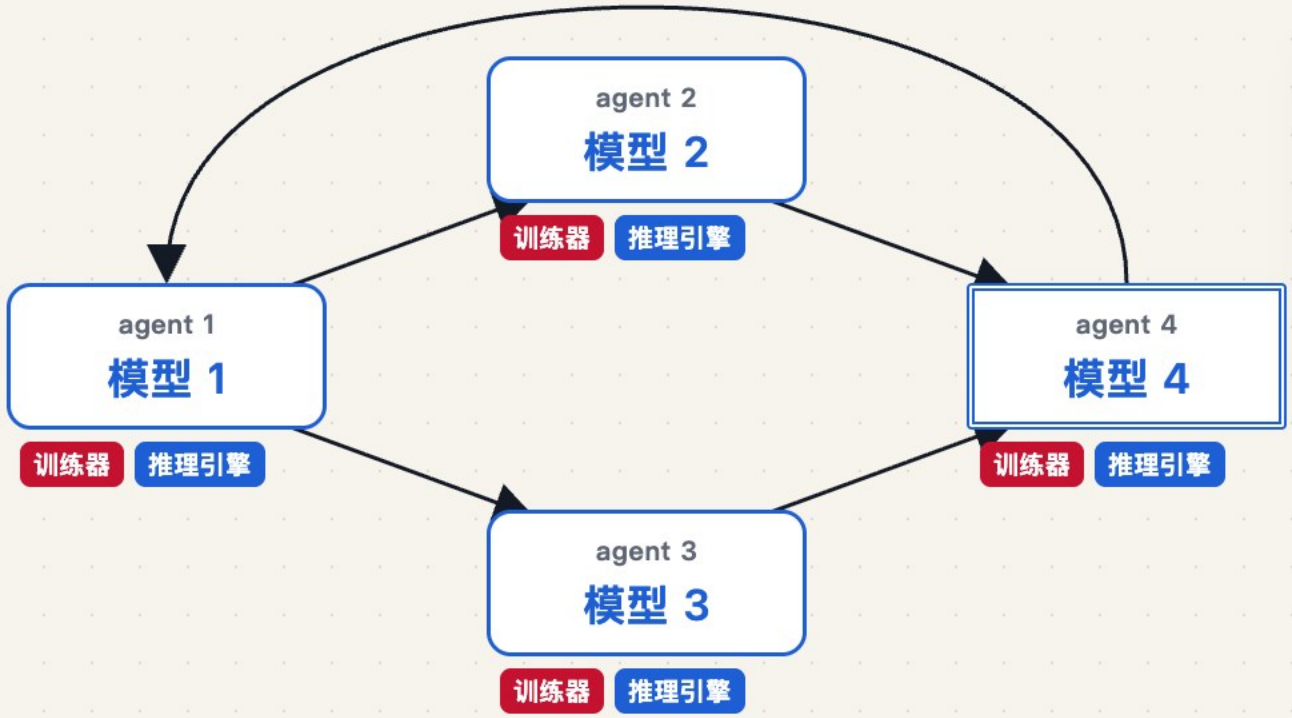
第一件：跨地区、用不同的卡训





课件 p.37（中文重绘）

第二件：同时训好几个模型



服务化之后，每个 agent：

可以用不同的权重

甚至不同的模型结构

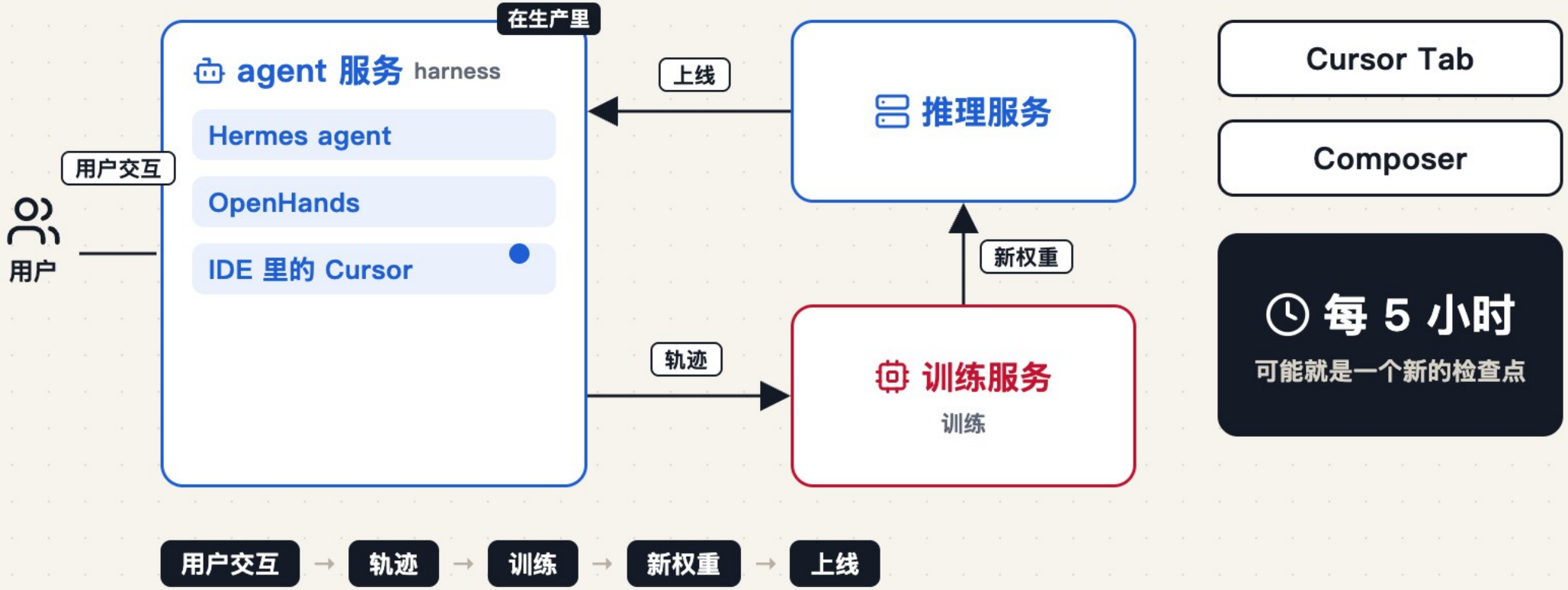
配不同的训练器、不同的推理引擎



课件 p.38（中文重绘）

总览
agent 强化学习的坑
系统设计

第三件：部署之后接着训（在线强化学习）



Cursor Tab

Composer

每 5 小时

可能就是一个新的检查点



课件 p.39（中文重绘）

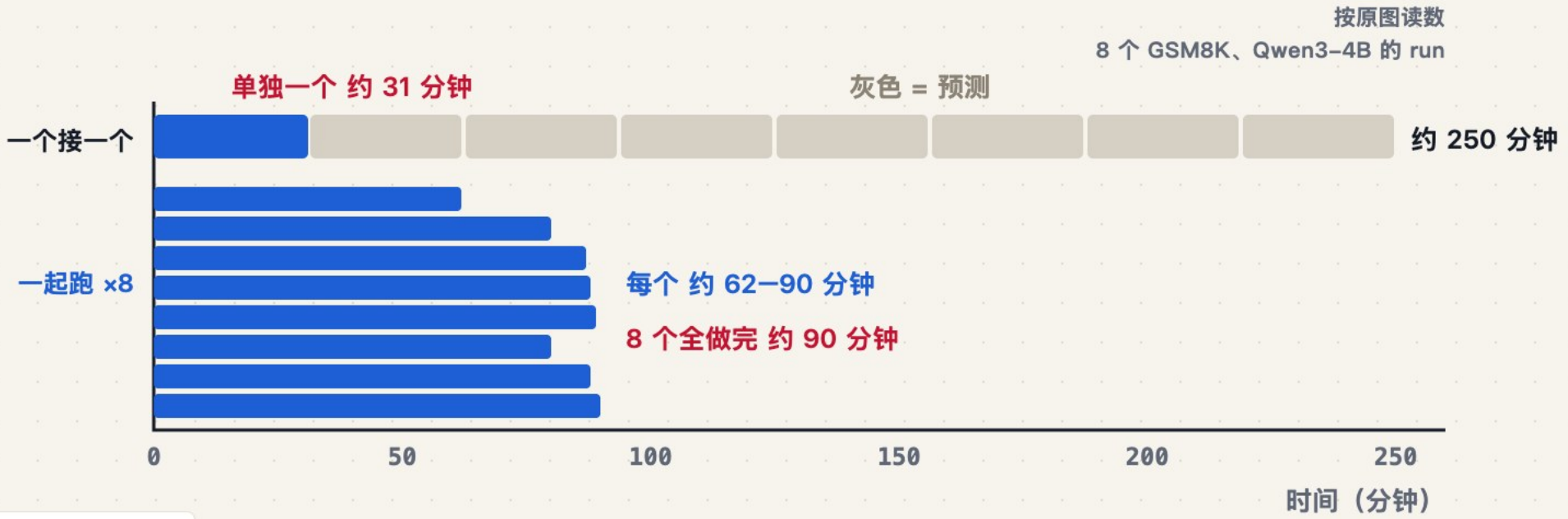
多租户、多 LoRA：一起跑，每个更慢，总体更快

托管训练服务

LoRA LoRA LoRA LoRA LoRA LoRA LoRA LoRA

LoRA LoRA LoRA LoRA LoRA LoRA LoRA LoRA

专门的算子 **Grouped-GEMM / SGMV**：同时对多份 LoRA 做 推理 前向 反向





课件 p.40（中文重绘）

Tinker：四个接口，把整套系统藏在后面

Tinker 托管训练服务

API 密钥 ••••••••

指定：模型 · 环境

~~把模型映射、切分到 GPU 上　自己找算力~~

训练客户端
每个训练任务一个 · 管一份正在训的 LoRA

采样客户端
每更新一次权重，拿一个新的



forward_backward
前向 + 反向，累积梯度

optim_step
按梯度更新权重

sample
生成 token

save_state
保存进度



课件 p.41（中文重绘）

Tinker：四个接口，把整套系统藏在后面

forward_backward
前向 + 反向，累积梯度

optim_step
按梯度更新权重

sample
生成 token

save_state
保存进度

代码写给 Tinker 的接口，还能换到 SkyRL Miles 这些开源实现，用自己的卡跑





课件 p.41（中文重绘）

强化学习系统：三块内容

01

总览

- 两套引擎各自优化
- 轨迹和权重同步来往

sample verify fwd bwd sync

一个训练步五格

02

agent 强化学习的坑

- 前缀保住就一直拼
- 传 token 不传字符串
- 太长用异步
- 复杂 harness 想好组批

Hel lo ≠ He llo

同一段字，token 切法对不上

03

系统设计

- 中间人代理
- 一切皆服务

控制器

一个控制器 + 一组服务



课件 p.42–43

排查用的指标表

指标	说明什么	危险信号
奖励 / pass@1	策略在训练题上有没有变好	长时间不动；或训练奖励涨、留出评测跌（奖励投机）
pass@k	覆盖：k 次里有没有一次能解出来	pass@1 涨它却跌：多样性在塌
熵	策略还在探索多少	突然塌向 0，或突然跳高（输出变乱）
importance_weight_max	最坏的 token 比值（训练 / 采样）	尖峰：不匹配或数据过时，少数 token 主导更新
梯度范数	截断前每次更新有多大	发散前出尖峰；一路往上漂
平均陈旧度	数据落后几个权重版本	慢慢变大：rollout 跟不上训练
训练端 MFU	训练的卡用得好不好	填充、负载不均或微批太小时会掉
计时	一步的时间花在 rollout、更新还是等待	等待大于 0：训练端在等 rollout



课件 p.42–43

CMU 11–768 · AI AGENTS

第 12 讲 · 强化学习系统

下一讲：第 13 讲 · Agent 安全

沙箱 · 凭证管理

每晚 20:00 更新



课件 p.42–43